FULL BUILD PLAN - OCTOBER 2026 - REPO REVIEW EDITION

AI Video Studio

Full plan after checking three open-source video editors - OpenChatCut, Palmier Pro and Pallaidium - against our Slot Studio-style product: topic and voiceover in, a finished, still-editable video, Shorts, thumbnails and publish pack out.

> **Verdict: build our own product. Do not fork any of the three into it.** All three carry GPL or AGPL licences, and Palmier Pro has already gone proprietary above its open-source release. What they give us is better than code: a proven architecture for the editable timeline, a proven agent interface, and a proven generation-backend pattern. We borrow the patterns, write our own code, and keep the product sellable.

What this plan does

- Reviews each repo on what it actually is - stack, features, maturity, requirements and licence - from its own README and file tree.
- Names exactly what to borrow, what to build ourselves, and what to skip from each one.
- Updates the earlier Slot Studio plan: the 9-stage pipeline stays, and gains an editable timeline, an agent (MCP) interface, and a swappable AI generation backend.
- Gives two build routes with honest timelines - fast fork vs. clean-room - and recommends one.

The one-line summary of each repo

Repo	What it is, in one line
OpenChatCut	Local-first, conversational AI video editor with a real multi-track timeline - the closest thing to our product core.
Palmier Pro	Swift-native macOS editor built for AI agents - the polish and agent-integration reference, not a code base for us.
Pallaidium	Generative AI movie studio inside Blender's video editor - the generation engine reference, from script to screen.

Sources checked 17 October 2026: github.com/0xsline/OpenChatCut, github.com/palmier-io/palmier-pro, github.com/tin2tin/Pallaidium. Star, commit and licence figures are as shown on those pages on that date and will move. Timelines are estimates for one developer building full-time with AI assistance, reusing our existing Whisper, ffmpeg and reels code.

Three repos, three different layers of the same product

Point	OpenChatCut	Palmier Pro	Pallaidium
Form	Web app + Electron desktop, local-first	Native macOS app only	Add-on inside Blender's Video Sequence Editor
Stack	React 19, TypeScript, Vite, Electron, Remotion, FFmpeg, WebGL	Swift, Metal kernels	Python, Blender, Diffusers / local AI models
Licence	AGPL-3.0	GPL-3.0 up to v0.7.6; later binaries proprietary	GPL-3.0; separate licences per AI model
Signals	About 2,400 stars, 1,080 commits, 28 releases; created July 2026	About 14,600 stars, 1,094 commits, 88 releases; created April 2026	About 1,500 stars, 819 commits, no formal releases; created March 2023
Needs	Node.js 24; runs on Mac, Windows, Linux	macOS 26 (Tahoe), Apple Silicon only	Blender 5.2+, Nvidia CUDA GPU with 6-16 GB VRAM, Windows-first
Agent link	Built-in agent + external agents over MCP	In-app agent + MCP server on localhost	MCP tools file + remote backend adapters
Best at	Editable timeline, transcript editing, captions	Native speed, editor polish, in-timeline generation	Generation breadth: image, video, voice, music, locally

How to read this table

These are not three competitors for the same slot. OpenChatCut answers **how does a user edit what the AI made** - nothing it generates is final; every edit lands on real tracks the user can undo. Palmier Pro answers **how good can an AI-native editor feel** - native performance, generation inside the timeline. Pallaidium answers **how do you generate every asset type in one place** - text, image, video, speech and music, locally on a GPU or through a remote backend.

Our product needs all three answers, in that order of importance. Editing first, because a one-shot generated video the user cannot fix is the weakness of the Slot Studio model. Generation second, because our MVP can start with user footage and licensed stock. Native polish last, because a web app is our distribution advantage - Windows and Mac, no install.

> **The pattern all three share:** an AI agent works on a real timeline through a defined tool interface (MCP - Model Context Protocol), and the human approves or undoes its edits. That convergence, from three independent projects, is the strongest signal in this review. We design for it from day one.

OpenChatCut is the blueprint for our editing core

OpenChatCut is an open-source alternative to the commercial ChatCut, built so that AI agents and a human edit the **same** project. Its core idea is the one our product was missing: the AI never outputs a finished, locked video. It writes clips, captions, transitions and effects onto a real multi-track timeline, where every change can be previewed, undone or re-edited by hand.

What it has that maps directly onto our plan

- **Timeline core:** multi-track video and audio, trim, split, ripple edits, keyframes, markers, undo and redo - held as immutable state with a command layer, independent of the UI and of any AI model.
- **Transcript-driven editing:** word-level transcription; delete a word or a pause in the text and the cut follows. Captions are linked to the transcript, styled, translatable, and exportable as SRT.
- **Motion graphics and effects:** editable templates, WebGL effects, LUTs, zooms and transitions - the same job our Remotion template library does, done their way.
- **Generation inside the editor:** image, video, speech, music and sound-effect jobs with progress tracking, plus online media search.
- **Exports:** MP4 with hardware H.264 acceleration where available, audio, captions, FCPXML for hand-off to other editors, and full project export.
- **Local-first data:** projects and media stay on the user's machine; API keys stay server-side; cloud features only switch on when configured.

The agent pattern worth copying exactly

External agents (Codex, Claude Code and others) connect over a Streamable HTTP MCP endpoint. An agent opens an **edit session**, makes its changes to an isolated draft, then submits it for review. In manual mode the user previews and approves or rejects the proposal inside the editor; approved changes land as a single undo step. Tools with side effects that cannot be rolled back - generation, export, deletion - are kept out of draft sessions. That is a safe, shippable design for "AI edits my video", and we adopt the same session-and-proposal model.

What we do not take

The code itself. OpenChatCut is AGPL-3.0 (see the licence page), its editor is a full professional timeline - far more scope than our MVP needs - and it is young and fast-moving, with unsigned early desktop builds. We take its architecture and interaction model, and build a smaller timeline that serves our pipeline: scene-based, template-driven, with free editing as an escape hatch rather than the main event.

Palmier Pro is the polish bar - and a licence warning

Palmier Pro is a video editor written from scratch in Swift for macOS, with generative AI inside the timeline and an MCP server that lets Claude, Codex or Cursor edit the same project as the in-app agent. Its stated north star is Premiere Pro, with AI integrated into the workflow rather than bolted on.

What it teaches us

- **Generation belongs on the timeline, not in a separate tool.** Palmier generates video and images with current models (its README names Seedance, Kling and Nano Banana Pro) as clips in the project, so a generated shot is edited like any other shot.
- **One MCP endpoint, many agents.** When the app is open it serves MCP on localhost; Claude Code, Codex, Cursor and Claude Desktop (via a bundled extension) all connect to the same surface. Our web app should expose the same kind of single, well-documented tool surface.
- **Native performance is a real advantage we will not have.** Swift and Metal give it scrubbing and rendering speed a browser editor will struggle to match. Our answer is different, not better: instant access on any OS, server-side rendering, and no install.

> **Why Palmier is reference only.** Its own README states that releases through v0.7.6, and source up to a marked point, were published under GPL-3.0 - later binary releases are proprietary and their source is not published, and the repo no longer accepts contributions. So the current product cannot be studied in code, forked, or learned from beyond that old snapshot. It also runs only on macOS 26 (Tahoe) on Apple Silicon, which excludes most of our users outright.

How we use it in the build

As a design and interaction benchmark during the evaluation spike: install the free build if a Mac is available, run one real edit with an agent connected over MCP, and note where its timeline and generation flow feel faster or clearer than OpenChatCut's. Those notes become requirements for our editor's feel - preview responsiveness, generation job states, agent proposal review - without a single line of its code or assets entering our product.

Palmier's history is also a business lesson for the roadmap page: a GPL project can be taken proprietary by its own authors for future versions, but a fork made by us would stay GPL forever. Building on it would cap what we can ever do with the product.

Pallaidium is the generation engine pattern

Pallaidium is a generative AI movie studio that lives inside Blender's Video Sequence Editor (VSE). Its workflow runs the full cycle in one place: start from lyrics or a scene description, turn it into a screenplay with shots, break it into timed strips on the timeline, generate the images, animate them, add voices and music, edit, and render. It can also run the cycle in reverse - captions out, screenplay reconstructed - to refine an idea before shooting real footage.

What it has that maps onto our plan

- **A complete generation matrix:** text to image, video, text, audio, music and speech; image to video and text; video to video, image and text - plus ControlNet-style controls, LoRA support, style selection, batch processing and an async render queue with job states and cancel controls.
- **A model-plugin system:** each model is a plugin with its own settings, and generated results land as strips (clips) on the timeline with their prompt and seed stored in the strip's metadata - so any shot can be re-run or varied. Our scene objects should store the same provenance.
- **A remote backend contract:** generation can be offloaded to any server speaking an OpenAI-style /v1 API dialect. Bundled adapters cover ComfyUI workflows, fal.ai cloud models and a custom URL, discovered through a model-list endpoint. This is exactly the swappable-backend design our architecture adopts.
- **Local-first generation:** diffusion image and video models, TTS with voice cloning and transcription all run on the user's own GPU when the hardware allows.

Why it is a backend pattern, not our product

The requirements are the reason: Blender 5.2 or later, an Nvidia CUDA GPU with 6-16 GB of VRAM, 20 GB or more of disk, and individual models of 6 GB or more (with tens of GB downloaded before the first run). That is a workstation tool for technical users - the opposite of our browser-in, video-out product. It is also GPL-3.0, and each AI model carries its own licence on top; its changelog notes, for example, a song model under a non-commercial licence. Any model we ever ship or call has to be licence-checked individually, the same way.

> **Where Pallaidium earns a place in our build:** as an optional power-user backend in v2. A creator with a GPU could connect their own local generation server - following the same /v1 contract - and our studio would send it generation jobs. We build the contract, not the Blender add-on.

The licence trap decides the build route

All three repos are copyleft. That single fact shapes this plan more than any feature list, so it gets its own page.

Repo	Licence as published	What that means for a paid, closed product
OpenChatCut	AGPL-3.0	The AGPL extends copyleft to network use: run a modified version as a web service and its source must be offered to its users. Forking it means our editor ships open-source.
Palmier Pro	GPL-3.0 to v0.7.6, then proprietary binaries, source unpublished	Only the old snapshot is usable, and anything built from it stays GPL. The current product is closed and off-limits as code.
Pallaidium	GPL-3.0, plus per-model licences	Distributing a product built on it triggers GPL source obligations; some bundled models are non-commercial, which would block paid use of those features entirely.

What stays fully legal and safe

- **Using all three as tools** for our own content work, unmodified or modified privately, without distributing them.
- **Studying their architecture and behaviour** and re-implementing the ideas in our own code - patterns, protocols and workflows are the lesson; copied code, assets and template files are the risk.
- **Speaking the same protocols:** MCP for agent tools and an OpenAI-style /v1 generation contract are open interfaces. Compatibility is not copying.
- **Calling permissively licensed libraries directly** - each dependency we adopt gets its own licence check at adoption time, recorded in the repo. This includes checking the current licence terms of our rendering stack (Remotion included) before commercial launch.

> **House rule for the build:** no code, template, prompt file or asset from any of the three repos enters our codebase. If a developer has read a repo's source for a feature, that feature is written from the behaviour spec in this plan, not from the source. This page is the record of that decision.

Borrow the patterns, build the core, skip the rest

Decision	Item and source
Borrow the pattern	Proposal-based agent editing: draft session, review, apply as one undo step (OpenChatCut). MCP as the single agent tool surface (all three). Generation jobs with states, progress and cancel (OpenChatCut, Pallaidium). Swappable generation backend behind an OpenAI-style /v1 contract, with ComfyUI and fal.ai-style adapters (Pallaidium). Provenance stored per clip: prompt, model, seed, source (Pallaidium strip metadata; OpenChatCut project data).
Build ourselves	Scene-based timeline editor (simpler than a pro NLE), Remotion motion-graphics template library, the pipeline orchestrator, transcript editing on faster-whisper word timings, caption system, render queue, publish pack (Shorts, thumbnails, metadata) - all carried over from the earlier plan and extended on the next page.
Use as-is, internally	OpenChatCut as a working editor for our own Pixalified content during the spike and while our editor is immature. Pallaidium on a GPU machine as a local generation testbed for evaluating models before we commit API spend.
Skip	Blender as a UI host. Native Swift / macOS-only anything. A full Premiere-class timeline (multi-cam, pro colour, FCPXML round-trip in MVP). Local diffusion models inside our web product. Palmier code of any version.

The updated pipeline: ten stages, one editable project



The change from the earlier plan is stages 6 and 7. Generation now writes **clips into a project** (footage, stock, or AI-generated, each with provenance), and a real - if deliberately small - **timeline stage** sits between assembly and render. The user, or an agent with approval, can re-cut, swap a shot, fix a caption or drop a scene there, instead of re-running the whole pipeline to change one shot. Approval gates stay where they were: script, storyboard, and preview before the master render.

Combined architecture: pipeline, timeline, agents, backends

Layer	Choice	Why, and what the repos changed
Frontend + editor	Next.js (React) + Tailwind; timeline as immutable state with a command layer	OpenChatCut's core lesson: timeline state independent of UI and AI, every change a command, undo built in from the start - not bolted on.
Agent interface	MCP server exposing draft edit sessions	All three repos converge on MCP. Built-in assistant and external agents (Claude Code, Codex) use the same tools; changes land as reviewed proposals.
Orchestrator	FastAPI (Python) job service; Redis + Celery queue	Unchanged from the earlier plan. Runs the 10 stages as resumable steps; generation and render are jobs with states, progress and cancel.
Speech	faster-whisper (self-hosted) + ffmpeg	Word timings drive transcript editing and captions - the OpenChatCut text-editing experience on our existing stack.
Generation backend	Internal /v1-style contract; adapters for cloud APIs first (image, video, TTS), ComfyUI adapter in v1	Pallaidium's contract pattern. Backends are swappable per job; every output records model, prompt, seed and cost.
Graphics + render	Remotion templates rendered headless; ffmpeg assembly and export	Templates stay data-driven JSON-to-component, as planned. Timeline preview uses the same compositions, so what the user scrubs is what renders.
Storage	S3 / Cloudflare R2 + Postgres	Project data is portable JSON (scenes, clips, transcript, provenance) - exportable as a full project file, the local-first lesson applied to a web app.

Design rules added by this review

- **Nothing the AI makes is final.** Every output lands as an editable project object. A generated video that cannot be fixed is a failed job, not a deliverable.
- **Side effects stay out of drafts.** Generation, export and deletion never run inside an agent draft session; they run only after approval, so a rejected proposal costs nothing.
- **Provenance on everything.** Facts keep their sources; clips keep their origin (user, stock, or generated - with model and prompt). The approval screens show both.
- **One tool surface.** If the built-in assistant can do it, an external agent can do it through MCP with the same limits. No private back doors.

Two routes: fast and open, or slower and sellable

Route	How	Cost of the route
A. Fork OpenChatCut	Take the AGPL editor as the base, add our pipeline stages and templates on top. MVP in an estimated 3-4 weeks.	The product is AGPL forever: source offered to every user of the service, no closed paid tier on that code base, and we inherit a large, fast-moving code base we did not write.
B. Clean-room (recommended)	Build our smaller timeline, orchestrator and templates ourselves, using the repos as behaviour references. MVP in an estimated 6-8 weeks.	Slower by roughly a month, and our editor will be narrower than OpenChatCut's for a long time. In exchange the product is ours: closed, sellable, licensable.

Roadmap on Route B

Phase	Estimate	Scope and exit test
0. Evaluation spike	5-7 days	Install OpenChatCut from source and run one real Pixalified video through it end to end; test Pallaidium on a GPU machine if available; note Palmier interactions if a Mac is available. Plus the earlier plan's CLI pipeline spike (research, script, Whisper cleanup, 3 Remotion graphics, ffmpeg assembly). Exit: one postable video, a written behaviour spec for our timeline and agent tools, and a keep / drop list per repo feature.
1. MVP	6-8 weeks	Web app: login, job creation, script and storyboard approval, voice upload + cleanup, transcript-based cutting, scene timeline (trim, swap, reorder, delete), 6 graphic templates, animated captions, generation via cloud adapters for image and TTS, 1080p render, 1 auto Short, title and description. Exit: 5 videos produced through the UI without developer help.
2. Sellable v1	+4-6 weeks	Payments and credits, MCP server with draft edit sessions, ComfyUI generation adapter, thumbnail generator (3 options), 2-3 Shorts per video, 12+ templates, per-channel brand kits, music library, project export. Exit: a paying creator completes a video start to finish, and an external agent edits a draft that the creator approves.
3. v2 polish	Ongoing	4K tier, Hindi / Hinglish tuning, team accounts, user-connected local generation backend (the Pallaidium pattern), template marketplace, deeper timeline features only where users actually ask.

Route A remains the fallback if speed ever matters more than ownership - for example, an internal-only tool that is never sold or hosted for outsiders. Internal use of the unmodified or privately modified AGPL editor does not force publication the way offering it as a service does; that distinction is what makes the spike legal and useful either way.

Costs, risks, and the decision to start

Costs - what changes versus the earlier plan

Hosting, render workers, storage and per-video LLM costs are as estimated in the earlier plan: low at early volume, dominated by renders. This review adds one new line - **generation credits**. Cloud image and video generation is the most expensive per-video item when used, priced per call by the provider and varying widely by model. It is therefore opt-in per scene, quoted to the user before the job runs, and charged as credits - never silently bundled into a flat plan. Local generation through a user's own backend (v2) costs us nothing and is a genuine selling point of the contract design.

Risks

Risk	Mitigation
Licence contamination	House rule on the licence page: no code, templates or assets from the three repos in our code base; behaviour specs only. Dependency licence check recorded for everything we adopt, before launch.
Scope creep into a Premiere clone	The timeline is scene-based and deliberately small. New editing features enter only when a real video needed them during the spike or MVP exit tests.
Generation cost and quality variance	Stock and user footage are the MVP default; generation is per-scene opt-in with an upfront credit quote, and outputs land as replaceable clips, not locked renders.
Clip copyright / YouTube terms	Unchanged from the earlier plan: licensed stock and user footage by default; any YouTube-clip feature is an explicit opt-in with the risk stated, never the default path.
Model licences (generation)	Every model offered is licence-checked individually before it is enabled - Pallaidium's own mix, including a non-commercial song model, shows why a blanket approval is unsafe.

Recommendation: approve the 5-7 day evaluation spike only

Do not approve the full build yet. The spike runs one real topic through OpenChatCut as-is and through our own CLI pipeline, tests generation backends, and returns three things: a postable video, a behaviour spec for our timeline and MCP tools, and firmer Phase 1 timelines. If the editable-timeline experience does not visibly beat the old one-shot pipeline on that video, we learn it in a week, not after the MVP.

Three decisions needed from you

- **Route:** B, clean-room (recommended), or A, fork for internal use only?
- **First user:** ourselves on Pixalified content first (recommended), or an external creator from the start?
- **MVP generation scope:** stock + own footage, with AI image and TTS only (recommended), or AI video generation in the MVP as well, at higher credit cost?

Sources: github.com/0xsline/OpenChatCut (AGPL-3.0; React 19, TypeScript, Electron, Remotion stack; MCP edit-session workflow), github.com/palmier-io/palmier-pro (GPL-3.0 through v0.7.6, later binaries proprietary; macOS 26, Apple Silicon; MCP server), github.com/tin2tin/Pallaidium (GPL-3.0; Blender 5.2+, CUDA GPU; generation matrix and remote backend contract) - all checked 17 October 2026. This plan follows and updates the earlier AI Video Studio build plan (Slot Studio comparison), whose pipeline, pricing shape and risk notes are carried forward where stated.